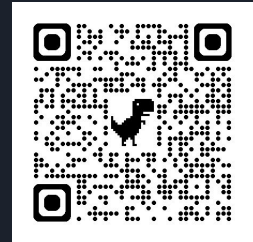




SPARQL-RL : a rules language for RDF

Product of the Data Shapes Working Group

2026-09-15





SPARQL-RL Task Force

- Robert David, Graphwise
- David Habgood, KurrawongAI
- Livio Robaldo, Swansea University
- Ognjen Savkovic, Free University of Bozen-Bolzano
- Simon Steyskal, Siemens AG
- Ted Thibodeau Jr, Openlink Software Inc
- Andy Seaborne, Apache Software Foundation



Why?

- Rules are knowledge
 - Predictable outcomes - publish rules for others to use on their data
- Use cases contributed to the Data Shapes WG
- Datalog because
 - It is well understood. Huge implementation experience.
 - Database centric
 - “Infer()” and “Query()”
 -
- Other evaluation systems may produce more answers
 - Answer Set Programming, N3, OWL-DL reasoners
- Not the first — N3, SWRL, RIF, RDF-SPIN, SHACL-AF(R), ...
 - Does not seem to be one universal “rules” for the semantic web layer



RDFS

Recursive Rules

```
## SubClassOf

RULE { ?x rdf:type ?type } WHERE { ?x rdf:type / rdfs:subClassOf ?type . }

RULE { ?x rdfs:subClassOf ?z }
WHERE { ?x rdfs:subClassOf / rdfs:subClassOf ?z }

## SubPropertyOf

RULE { ?x ?p ?y } WHERE { ?x ?q ?y . ?q rdfs:subPropertyOf ?p }

RULE { ?x rdfs:subPropertyOf ?z }
WHERE { ?x rdfs:subPropertyOf ?y . ?y rdfs:subPropertyOf ?z }

## Domain

RULE { ?x rdf:type ?type } WHERE { ?p rdfs:domain ?type . ?x ?p ?any }

## Range

RULE { ?z rdf:type ?type } WHERE { ?p rdfs:range ?type . ?any ?p ?z }
```

Weighted Path Example

```
## Condition for one step in the path.
RULE { ?x :connected ?z } WHERE {
    ?x :link ?z { | :weight ?w | }
    FILTER(?w > 5)
}

## Recursion "connected"
RULE { ?x :connected ?y } WHERE {
    ?x :connected ?z . ?z :connected ?y
}
```

```
## What is connected to :x5?
RULE { ?x :result "path to x5" }
WHERE { ?x :connected :x5 }
```

Inferred:

```
:x3 :result "path to x5" .
:x4 :result "path to x5" .
```

```
PREFIX : <http://example/>
```

```
:x0 :link :x1 { | :weight 9 | } .
```

```
:x1 :link :x2 { | :weight 9 | } .
```

```
# Weak link: :x2 to :x3
```

```
:x2 :link :x3 { | :weight 1 | } .
```

```
:x3 :link :x4 { | :weight 8 | } .
```

```
:x4 :link :x5 { | :weight 8 | } .
```



Negation-as-Failure

```
RULE { ?x rdf:type :UnclassifiedSize }  
WHERE {  
  ?x rdf:type :Place .  
  NOT { ?x :population ?p }  
}
```

Need to wait until all possible " ?x :population ?p " have been found or inferred



RDF Features

A rule that runs exactly once after its pre-conditions are ready

Never recursive

- Assignment
- Blank nodes in the head template
- Avoid partial rule conclusions
 - Requirements for being “well-formed”

```
## Infinite
```

```
RULE { ?x :value ?v1 }  
WHERE {  
    ?x :value ?v  
    SET ( ?v1 := ?v + 1 )  
}
```

```
## Infinite
```

```
RULE { [ ] :p :o }  
WHERE {  
    ?s :p :o .  
}
```



Stratification

1. Calculate dependency graph
 - Label as “closed” if negation, or run-once
 - Else label as “open” dependency
2. Check there are no recursive loops with a “closed” link (error -> can not stratify)
3. Split rules into groups (“strata”) where no rule has a “closed” relation to other rules in the group
4. Make into a sequence of pairs – “run-once” and “general” (including recursive), starting with base layer of rules with no dependencies



Execution

Apply stratification to get a sequence of (run-once, general) rule pairs.

For each pair in sequence order:

- Execute run-once

- Execute general rules until no new triples (“fix-point”)



Futures?

- Same syntax, more expressive implementations:
 - ASP, Theorem provers
 - What about execution planning involving knowledge of the data?
- Useability
 - Syntax for patterns of use e.g. property paths with conditional steps
 - TRANSITIVE / SYMMETRIC / INVERSE
 - Synonyms
- Features
 - Aggregates - at least counting
 - Tuples, not just triples
- Usage
 - Published rule sets for entailment with a data focus
 - Blank nodes under control!
 - with/without datatype entailments
- Transformation Rules
 - ? SPARQL-RL and SPARQL Update



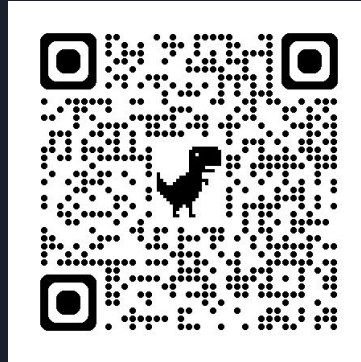
Implementations

All WIP!

- [Eyeleng](#) – from Gent!
- [Comunica](#) – from Gent!
- [shacl-rules.app.dev.kurrawong.ai](#)
- [SHACL Rules Playground](#)
- [jena-rules](#)



SPARQL 1.2 RL

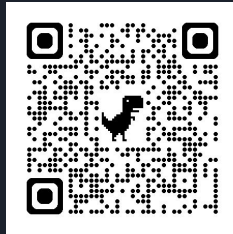


<https://www.w3.org/TR/sparql12-rl/>





Slides





SPARQL-RL

- Specify outcomes across implementations
 - Specification - gives algorithms
 - Test suite
- Forwards or Backwards execution
- Compile to SPARQL pattern matching
 - Use the SPARQL function library
 - One possible route to scale and efficiency